Proposition de résolution du problème de coloration de somme minimale (Minimum Sum Coloring Problem) par recuit simulé

Quentin HÛ
Métaheuristiques – Université d'Angers

6 octobre 2026

Table des matières

1 Introduction **2**
1.1 Le problème de coloration de somme minimale (MSCP) . . . 2
1.2 Définition formelle . . . 2
1.3 Quelques résultats dans la littérature . . . 2
1.4 Objectifs du projet et plan du rapport . . . 3

2 État de l'art **4**
2.1 Méthodes exactes . . . 4
2.2 Métaheuristiques . . . 4
2.3 Positionnement de ce travail . . . 4

3 Méthodologie développée **5**
3.1 Structures de données utilisées . . . 5
3.2 Phase 1 : construction d'une solution initiale par DSATUR . . . 5
3.3 Phase 2 : amélioration par recuit simulé . . . 5
3.3.1 Rappel du principe du recuit simulé . . . 5
3.3.2 Évaluation d'une solution et pénalisation des conflits . . . 6
3.3.3 Structures de voisinage . . . 6
3.3.4 Calibration du schéma de refroidissement . . . 8
3.3.5 Algorithme complet du recuit simulé . . . 9

4 Résultats et analyse **10**
4.1 Paramètres et critère d'arrêt . . . 10
4.2 Résultats du recuit simulé . . . 10
4.3 Multi-start . . . 11
4.4 Visualisation du recuit simulé . . . 12
4.5 Visualisation du multi-start . . . 13
4.6 Analyse des résultats . . . 13

5 Conclusion et perspectives **14**

A Annexes **16**
A.1 Recuit simulé one-move à évaluation incrémentale . . . 16
A.2 Évolution de la fonction objectif par instance . . . 17

1 Introduction

1.1 Le problème de coloration de somme minimale (MSCP)

Le problème de coloration de graphe classique (Graph Coloring Problem : GCP) est le suivant : à partir d'un graphe non orienté $G = (V, E)$, il faut trouver une coloration telle que deux sommets adjacents aient des couleurs différentes, en minimisant le nombre total de couleurs utilisées. Le GCP est un problème NP-Complet de référence et possède de très nombreuses applications. Une variante du GCP est le Minimum Sum Coloring Problem (MSCP). Au lieu de minimiser le nombre de couleurs, on associe pour chacune d'entre elles un entier, on minimise maintenant la somme des indices des couleurs attribués à chaque sommet. Ainsi, une telle coloration optimale n'est pas nécessairement optimale au regard d'une coloration classique [9]. Plusieurs applications concrètes existent [5].

1.2 Définition formelle

Le MSCP peut se traduire de manière formelle de la manière suivante : Étant donné $G = (V, E)$ un graphe non orienté, trouver une coloration $c : V \to \{1, \ldots, k\}$ telle que $c(u) \neq c(v)$ pour toute arête $(u, v) \in E$, minimisant $\sum_{v \in V} c(v)$

Où :

— $x_{v,k} \in \{0, 1\}$ vaut 1 si le sommet v reçoit la couleur k, 0 sinon

— K : nombre maximum de couleurs autorisées (borne supérieure, ex. $|V|$)

$$\min \quad \sum_{v \in V} \sum_{k=1}^{K} k \cdot x_{v,k} \tag{1}$$

$$\text{s.c.} \quad \sum_{k=1}^{K} x_{v,k} = 1, \qquad \forall v \in V \tag{2}$$

$$x_{u,k} + x_{v,k} \leq 1, \qquad \forall (u, v) \in E,\ \forall k \in \{1, \ldots, K\} \tag{3}$$

$$x_{v,k} \in \{0, 1\}, \qquad \forall v \in V,\ \forall k \in \{1, \ldots, K\} \tag{4}$$

1.3 Quelques résultats dans la littérature

Voici une liste d'instances qu'on utilisera pour tester notre algorithme. Les résultats connus sont issus de la littérature [5]. Le tableau 1 présente pour chacune de ces instances le nombre de sommets $|V|$, le nombre d'arêtes $|E|$ et les meilleures bornes connues (lower bound et upper bound) pour le MSCP.

TABLE 1 – Instances de test et meilleures bornes connues pour le MSCP

Instance	$\|V\|$	$\|E\|$	lb	ub	Optimum prouvé
queen5.5	25	160	75	75	oui
queen7.7	49	476	196	196	oui
queen8.12	96	1368	624	624	oui
huck	74	301	243	243	oui
games120	120	638	442	443	non
anna	138	493	273	276	non
david	87	406	234	237	non
myciel4	23	71	34	45	non
le450.25c	450	17343	3657	4515	non
DSJC500.5	500	62624	2923	10886	non
DSJC1000.9	1000	449449	26557	103445	non

1.4 Objectifs du projet et plan du rapport

L'objectif de ce projet est donc de développer une métaheuristique permettant de résoudre le MSCP. Ici l'idée est de proposer une approche claire avec une implémentation d'une métaheuristique conforme à sa formulation théorique, sans modifications excessives. Pour cela j'ai choisi de développer une approche en deux étapes inspirée des travaux de Jin, Yan and Hamiez, Jean-Philippe et Hao, Jin-Kao [5]. En premier temps une construction gloutonne par DSATUR pour obtenir une solution initiale, puis une amélioration de cette solution par recuit simulé. On comparera ensuite deux méthodes de voisinages différentes.

Nous verrons donc d'abord en détail la méthodologie développée, puis le protocole expérimental mis en place pour tester l'algorithme sur des instances de référence, et enfin les résultats obtenus et leur analyse.

2 État de l'art

Le MSCP est un problème NP-difficile [9]. Pour un graphe quelconque on recourt en pratique à des méthodes exactes sur les petites instances et à des métaheuristiques sur les autres [5].

2.1 Méthodes exactes

La formulation en programmation linéaire en nombres entiers (PLNE) donnée plus haut en 1.2 est la plus directe, mais sa relaxation est faible et sa taille croît comme $|V| \times K$. Lecat, Lucet et Li ont comparé quatre méthodes exactes (branch & bound, MaxSAT/MinSAT et programmation par contraintes) [10]. Minot, Ndiaye et Solnon combinent PLNE et programmation par contraintes [11]. Plus récemment, Delle Donne et al. ont proposé le premier algorithme de *branch-and-price* pour le MSCP, fondé sur une formulation résolue par génération de colonnes. Il permet de prouver l'optimalité d'une partie des instances DIMACS restées ouvertes [4]. Ces méthodes restent toutefois limitées aux graphes de taille modeste : sur les instances de grande taille ou denses, seules des bornes inférieures sont disponibles (voir les écarts $lb < ub$ du tableau 1).

2.2 Métaheuristiques

Pour les instances de taille réaliste, la littérature s'appuie sur des heuristiques constructives (gloutons de type DSATUR adaptés à la somme) puis sur des métaheuristiques à base de recherche locale. Voici quelques approches [5] :

- l'algorithme heuristique de Wu et Hao (2012) fondé sur l'extraction d'ensembles indépendants [12] ;
- la *Breakout Local Search* (BLS) de Benlic et Hao (2012), qui alterne recherche locale et perturbations adaptatives [1] ;
- l'algorithme mémétique de Jin, Hao et Hamiez (2014), qui associe une recherche tabou à deux voisinages et un croisement multi-parents [7] ;
- la recherche évolutionnaire hybride de Jin et Hao (2016), qui fournit à la fois des bornes supérieures et inférieures [6].

Ces algorithmes, souvent hybrides (population + recherche locale), fournissent la plupart des meilleures bornes supérieures connues sur les graphes DIMACS et COLOR [5]. C'est aussi de cette revue que proviennent les valeurs de référence utilisées dans ce rapport.

2.3 Positionnement de ce travail

Le recuit simulé a une longue histoire en coloration de graphes [3]. l'objectif est donc maintenant de l'appliquer sur un MSCP. L'approche étudiée ici (DSATUR puis recuit simulé, avec deux voisinages simples) est plus élémentaire que les algorithmes ci-dessus : elle ne comporte ni population, ni croisement, ni mécanisme de perturbation. Elle peut par contre servir de base de comparaison pour mesurer ce qu'un recuit « pur » peut atteindre.

3 Méthodologie développée

3.1 Structures de données utilisées

Dans la suite de ce rapport, je présenterai les différents algorithmes développés pour résoudre le MSCP. On manipulera donc différentes structures de données et variables pour représenter les graphes et les colorations, voici un rappel des principales :

- n : nombre de sommets du graphe.
- x ou $colorOf$: vecteur de taille n représentant une coloration, où $x[v]$ (ou $colorOf[v]$) est la couleur attribuée au sommet v.
- $adjOf$: liste d'adjacence du graphe, où $adjOf[v]$ est l'ensemble des voisins du sommet v.
- $edges$: liste des arêtes du graphe, où chaque élément est un couple (u, v) représentant une arête entre les sommets u et v.

3.2 Phase 1 : construction d'une solution initiale par DSATUR

DSATUR [2] est un algorithme glouton, le principe est le suivant : On attribue à chaque sommet non coloré un "degré de saturation" $dsat(v)$. Ce degré est défini comme le nombre de couleurs distinctes déjà utilisées par les voisins de v. À chaque étape on colorie le sommet non coloré ayant le plus grand degré de saturation (en cas d'égalité, on prend simplement le premier sommet rencontré). La nouvelle couleur attribuée est la plus petite couleur disponible.

Voici une version pseudo-code pour DSATUR.

Algorithm 1 DSATUR (`glouton.jl`)

$colorOf[v] \leftarrow 0$ pour tout $v \in V$
$UnColored \leftarrow V$
while $UnColored \neq \emptyset$ **do**
    $v^\star \leftarrow \arg\max_{v \in UnColored} \text{dsat}(v)$
    $colorOf[v^\star] \leftarrow \min\{c \geq 1 : c \notin \{colorOf[u] : u \in adjOf(v^\star)\}\}$
    $UnColored \leftarrow UnColored \setminus \{v^\star\}$
end while
return $colorOf$

3.3 Phase 2 : amélioration par recuit simulé

3.3.1 Rappel du principe du recuit simulé

Le recuit simulé tient son origine d'une analogie physique avec le recuit métallurgique [8]. Le principe général est le suivant : à partir d'une solution courante x, tirer un voisin x', calculer $\Delta = f(x') - f(x)$, Accepter systématiquement x' comme nouvelle solution courante si $\Delta \leq 0$, sinon accepter avec probabilité $e^{-\Delta/T}$ (critère de Metropolis).

$$P_{T,x}(\text{accepter } x') = \begin{cases} 1 & \text{si } \Delta \leq 0 \\ e^{-\Delta/T} & \text{si } \Delta > 0 \end{cases} \quad , \Delta = f(x') - f(x)$$

Le paramètre T joue le rôle de la "température". Plus celle-ci est élevée, plus le recuit simulé accepte des solutions dégradantes, il est donc propice à l'exploration de l'espace des solutions. Au fur et à mesure de l'exécution, la température descend et la probabilité d'accepter une solution dégradante diminue, favorisant l'exploitation de la solution courante.

3.3.2 Évaluation d'une solution et pénalisation des conflits

Soit x une solution (coloration), on peut définir la fonction objectif $f(x)$ comme suit :

$$f(x) = \sum_{i \in V} x_i$$

On introduira une pénalisation des conflits. En effet, lors de l'exploration nous accepterons des solutions parfois non valides (avec des sommets adjacents de même couleur). Ainsi, cette non validité doit être pénalisée dans la fonction objectif pour permettre de guider le recuit simulé vers des solutions réalisables. On définit donc la fonction objectif pénalisée comme suit :

$$f_p(x) = \sum_{i \in V} x_i + p \cdot (\text{nombre d'arêtes en conflit})$$

On pourra prendre $p = n + 1$ pour garantir qu'une seule violation de contrainte coûte plus cher que n'importe quel gain possible sur la somme des couleurs.

3.3.3 Structures de voisinage

Beaucoup de structures de voisinage sont possibles pour le MSCP [5]. Certains voisinages garantissent de ne pas créer de conflits, d'autres permettent de créer des conflits pour explorer plus largement l'espace des solutions et certains utilisent les propriétés du graphe (extraction d'ensembles indépendants, etc.). Nous verrons ici deux voisinages, 'one-move' et 'exchange-swap'.

One-move Un premier voisinage, le plus simple, est le "one-move". À partir d'une solution courante x, on tire un sommet v au hasard parmi les sommets en conflit, ou parmi tous les sommets si aucun n'est en conflit, enfin on lui attribue une nouvelle couleur c tirée uniformément dans l'ensemble des couleurs disponibles (sans retirer celle déjà utilisée) plus une couleur supplémentaire (pour permettre d'augmenter le nombre de couleurs si nécessaire).

Algorithm 2 voisinage (tirage d'un voisin de la solution courante)

```
1:  function VoisinageOneMove(colorOf, adjOf)
2:      ConflictVertices ← ∅
3:      for each v ∈ V do
4:          for each u ∈ adjOf[v] do
5:              if colorOf[v] = colorOf[u] then
6:                  ConflictVertices ← ConflictVertices ∪ {v}
7:                  break                          ▷ On passe au sommet v suivant
8:              end if
9:          end for
10:     end for
11:     if ConflictVertices ≠ ∅ then
12:         v ← tirage aléatoire uniforme dans ConflictVertices
13:     else
14:         v ← tirage aléatoire uniforme dans V
15:     end if
16:     newColor ← tirage aléatoire uniforme dans {1, ..., max(colorOf) + 1}
17:     x' ← colorOf
18:     x'[v] ← newColor
19:     return x'
20: end function
```

Ainsi, même si l'on choisit en priorité les sommets avec conflit, rien ne garantit que la nouvelle solution ait moins de conflits que la solution précédente. On autorise donc bien une exploration large, régulée par la température T et la pénalisation p dans la fonction objectif.

La version de base du voisinage one-move ci-dessus recalcule la fonction objectif f_p sur toutes les arêtes et rescanne tous les sommets pour trouver ceux en conflit à chaque itération, soit un coût en $O(|E|)$ par itération. L'algorithme optimisé donne le même résultat en ne regardant que les voisins du sommet déplacé, soit un coût en $O(\deg(v))$.

Changer la couleur de v de c_{old} à c_{new} ne modifie que les arêtes incidentes à v :

$$\Delta = f_p(x') - f_p(x) = (c_{new} - c_{old}) + p\,\big(a_{new} - a_{old}\big),$$

où a_{old} (resp. a_{new}) est le nombre de voisins de v de couleur c_{old} (resp. c_{new}). On maintient donc, d'une itération à l'autre, le nombre $conf[v]$ de voisins de même couleur que v (qui donne a_{old} sans calcul), l'ensemble C des sommets en conflit (stocké dans un tableau avec la position de chaque sommet, ce qui permet insertion, suppression et tirage uniforme en $O(1)$), et la couleur maximale $c_{\max}$ (grâce à un compteur du nombre de sommets par couleur). L'algorithme, trop long pour être détaillé ici, est disponible en annexe A.1.

Exchange Swap Ce voisinage consiste à sélectionner un sous-ensemble de sommets $A \subset V_i$ ($|A| > 1$) et un autre sous-ensemble de sommets $B \subset V_j$ ($|B| > 1$), avec V_i l'ensemble des sommets de couleur i et $i \neq j$, tels que le sous-graphe induit par $A \cup B$ soit une composante connexe. Les sommets de A et de B sont ensuite échangés entre les deux groupes [5]. La nouvelle solution x' est réalisable si la solution initiale x est elle-même réalisable (et inversement). Sachant que notre solution initiale, sortie de DSATUR, est réalisable, alors ce type de voisinage explorera uniquement des solutions réalisables.

Cette méthode tient son origine du concept des chaînes de Kempe, et a été introduite comme un outil de recherche locale par Chams, Hertz & de Werra (1987) [3].

Algorithm 3 composante_connexe (extraction de la composante bicolore $\{c_1, c_2\}$)

```
1: function ComposanteConnexe(v_0, c_1, c_2, colorOf, adjOf)
2:     Attente ← {v_0}
3:     Result ← {v_0}
4:     while Attente ≠ ∅ do
5:         v ← un élément de Attente
6:         for each u ∈ adjOf[v] do
7:             if colorOf[u] ∈ {c_1, c_2} et u ∉ Result then
8:                 Attente ← Attente ∪ {u}
9:             end if
10:        end for
11:        Result ← Result ∪ {v}
12:        Attente ← Attente \ {v}
13:    end while
14:    return Result
15: end function
```

Algorithm 4 voisinage_swap (échange des couleurs)

```
1: function VoisinageSwap(colorOf, adjOf)
2:     v_0 ← tirage aléatoire uniforme d'un sommet
3:     c_1 ← colorOf[v_0]
4:     c_2 ← colorOf[u] avec u tiré aléatoirement dans adjOf[v_0]
5:     Composante ← ComposanteConnexe(v_0, c_1, c_2, colorOf, adjOf)
6:     x' ← colorOf
7:     for each v ∈ Composante do
8:         if colorOf[v] = c_1 then
9:             x'[v] ← c_2
10:        else
11:            x'[v] ← c_1
12:        end if
13:    end for
14:    return x'
15: end function
```

3.3.4 Calibration du schéma de refroidissement

L'idée maintenant est de calibrer les valeurs de départ T_0 et finale $T_{\min}$ de la température pour chaque instance. Pour cela, on regarde la dégradation moyenne $\overline{\Delta_{\text{dégrad}}}$ observée sur un échantillon de mouvements *voisinage*() depuis la solution gloutonne.

$$T_0 = \frac{-\overline{\Delta_{\text{dégrad}}}}{\ln(p_{\text{init}})} \qquad T_{\min} = \frac{-\overline{\Delta_{\text{dégrad}}}}{\ln(p_{\text{fin}})}$$

On choisit T_0 tel qu'une dégradation moyenne soit acceptée avec une probabilité p_{init} en début de recherche. De la même manière, $T_{\min}$ est calculé pour autoriser les modifications avec une probabilité p_{fin} en fin de recherche. Entre les deux, la température (et donc la probabilité d'acceptation) diminue de manière géométrique :

$$T_{i+1} = T_i * \alpha$$

Pour un nombre de paliers de température K, on a $\alpha = (T_{\min}/T_0)^{(1/K)}$. Dans l'implémentation, $p_{\text{init}} = 0.5$ et $p_{\text{fin}} = 0.0001$. La calibration utilise le voisinage qui est concerné et n'applique pas de pénalité lors de l'évaluation.

3.3.5 Algorithme complet du recuit simulé

Algorithm 5 Recuit simulé

$x \leftarrow x_0$ ▷ solution initiale issue de DSATUR
$f_{\text{courant}} \leftarrow f(x), \quad x_{\text{best}} \leftarrow x, \quad f_{\text{best}} \leftarrow f_{\text{courant}}$
$T \leftarrow T_0$
while $T > T_{\min}$ **do**
    for $i = 1$ **à** nbIter **do**
        $x' \leftarrow \text{VOISINAGE}(x)$
        $\Delta \leftarrow f(x') - f_{\text{courant}}$
        if $\Delta \leq 0$ **ou** $\text{rand}() < e^{-\Delta/T}$ **then**
            $x \leftarrow x', \quad f_{\text{courant}} \leftarrow f(x')$
            if $f_{\text{courant}} < f_{\text{best}}$ **then**
                $x_{\text{best}} \leftarrow x, \quad f_{\text{best}} \leftarrow f_{\text{courant}}$
            end if
        end if
    end for
    $T \leftarrow \alpha \cdot T$
end while
return $x_{\text{best}}, f_{\text{best}}$

4 Résultats et analyse

4.1 Paramètres et critère d'arrêt

Voici les paramètres retenus pour notre expérimentation. Le recuit simulé est en théorie capable d'aller chercher de meilleures solutions en augmentant la durée de l'exécution. Mais il a fallu calibrer les paramètres et limiter le nombre d'itérations pour obtenir un bon compromis entre qualité de solution et temps de calcul raisonnable. Cela dépend bien sûr aussi de la puissance de calcul de l'ordinateur utilisé.

TABLE 2 – Paramètres du recuit simulé retenus pour les expérimentations

Paramètre	Valeur	Rôle
p	$n+1$	Pénalité par arête en conflit
K	1500	Nombre de paliers de température
nbIter	600	Itérations par palier de température
p_{init}	0.5	Probabilité moyenne d'acceptation à T_0
p_{fin}	0.0001	Probabilité moyenne d'acceptation à $T_{\min}$

4.2 Résultats du recuit simulé

Les tableaux suivants présentent le résultat du recuit simulé sur un ensemble d'instances de la littérature [5]. Pour chaque instance on lance l'algorithme 10 fois avec des graines aléatoires différentes. On affiche ensuite dans les tables 3 et 4 les résultats statistiques obtenus pour chaque instance, soit :

- *Instance* : Le nom de l'instance testée.
- *ub* : La meilleure borne supérieure connue pour cette instance.
- *DSATUR* : La valeur de la solution initiale construite par DSATUR.
- *Min* : La meilleure solution obtenue sur les 10 exécutions.
- *Moyenne* : La moyenne des solutions obtenues sur les 10 exécutions.
- σ : L'écart-type des solutions obtenues sur les 10 exécutions.
- *Écart min (%)* : L'écart relatif entre la meilleure solution obtenue et la borne supérieure connue.
- *Écart moy. (%)* : L'écart relatif entre la solution moyenne obtenue et la borne supérieure connue.
- *Temps moy. (s)* : Le temps moyen d'exécution sur les 10 exécutions.

Pour obtenir des résultats comparables, les mêmes paramètres de recuit simulé sont utilisés pour les deux voisinages. Néanmoins, le voisinage *one-move* est beaucoup moins coûteux en calcul que le voisinage *swap*. Il peut donc être utilisé avec des valeurs de K et *nbIter* beaucoup plus élevées pour obtenir de meilleurs résultats.

TABLE 3 – Résultats du recuit simulé (*one-move*) sur plusieurs exécutions

Instance	ub	DSATUR	Min	Moyenne	σ	Écart min (%)	Écart moy. (%)	Temps moy. (s)
queen5.5	75	87.0	75	75.0	0.0	0.0	0.0	0.1
queen7.7	196	243.0	196	212.0	9.3	0.0	8.16	0.1
queen8.12	624	639.0	629	632.8	2.9	0.8	1.41	0.1
huck	243	304.0	243	244.8	1.0	0.0	0.74	0.1
games120	443	462.0	457	459.6	1.9	3.16	3.75	0.1
anna	276	352.0	290	294.4	3.4	5.07	6.67	0.1
david	237	292.0	247	251.8	3.6	4.22	6.24	0.1
myciel4	45	55.0	45	45.0	0.0	0.0	0.0	0.1
le450.25c	4515	5708.0	5436	5499.4	41.8	20.4	21.8	0.2
DSJC500.5	10886	15371.0	14830	14980.1	84.3	36.23	37.61	0.5
DSJC1000.9	103445	145020.0	145020	145020.0	0.0	40.19	40.19	4.5

TABLE 4 – Résultats du recuit simulé (*swap*) sur plusieurs exécutions

Instance	ub	DSATUR	Min	Moyenne	σ	Écart min (%)	Écart moy. (%)	Temps moy. (s)
queen5.5	75	87.0	78	78.0	0.0	4.0	4.0	1.2
queen7.7	196	243.0	202	202.2	0.4	3.06	3.16	1.9
queen8.12	624	639.0	628	628.7	0.8	0.64	0.75	2.6
huck	243	304.0	246	252.9	6.9	1.23	4.07	1.0
games120	443	462.0	444	446.3	1.3	0.23	0.74	1.8
anna	276	352.0	343	349.4	3.1	24.28	26.59	1.5
david	237	292.0	280	290.3	3.8	18.14	22.49	1.3
myciel4	45	55.0	45	45.0	0.0	0.0	0.0	1.2
le450.25c	4515	5708.0	5539	5670.0	50.9	22.68	25.58	7.4
DSJC500.5	10886	15371.0	13298	13503.0	118.7	22.16	24.04	7.7
DSJC1000.9	103445	145020.0	128143	129057.6	676.3	23.88	24.76	11.6

4.3 Multi-start

La rapidité des voisinages *one-move* et *swap* nous permet de mettre en place ce que j'appelle ici un multi-start. L'idée est de lancer plusieurs exécutions du recuit simulé à partir de solutions initiales différentes. On construit donc un glouton *randDSATUR* qui choisit aléatoirement le sommet à colorier parmi les 10% des sommets ayant le plus grand degré de saturation. On lance ensuite le recuit simulé à partir de cette solution initiale. Le *one-move* étant très rapide, on pourra lancer 100 exécutions pour chaque instance, on en lancera seulement 10 pour le *swap*.

TABLE 5 – Résultats du multi-start *one-move*

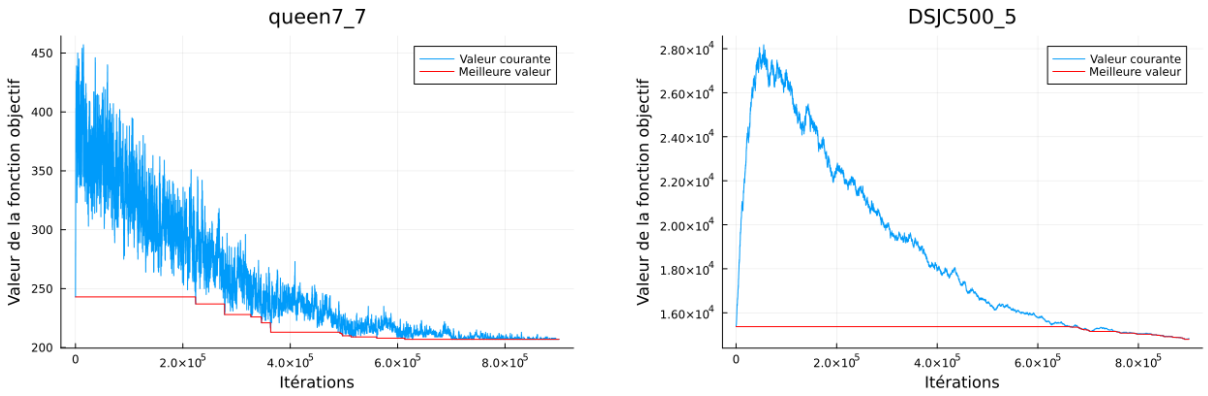
Instance	ub	Meilleure solution sur 100 starts	Écart à ub (%)
queen5.5	75	75	0%
queen7.7	196	196	0%
queen8.12	624	625	0.16%
huck	243	243	0%
games120	443	447	0.9%
anna	276	281	1.81%
david	237	240	1.27%
myciel4	45	45	0%
le450.25c	4515	5375	19.05%
DSJC500.5	10886	14447	32.71%
DSJC1000.9*	103445	140144	35.48%

TABLE 6 – Résultats du multi-start *swap*

Instance	ub	Meilleure solution sur 10 starts	Écart à ub (%)
queen5.5	75	75	0%
queen7.7	196	199	1.53%
queen8.12	624	627	0.48%
huck	243	243	0%
games120	443	443	0%
anna	276	294	6.52%
david	237	238	0.42%
myciel4	45	45	0%
le450.25c	4515	4978	10.25%
DSJC500.5	10886	12681	16.49%
DSJC1000.9	103445	126339	22.13%

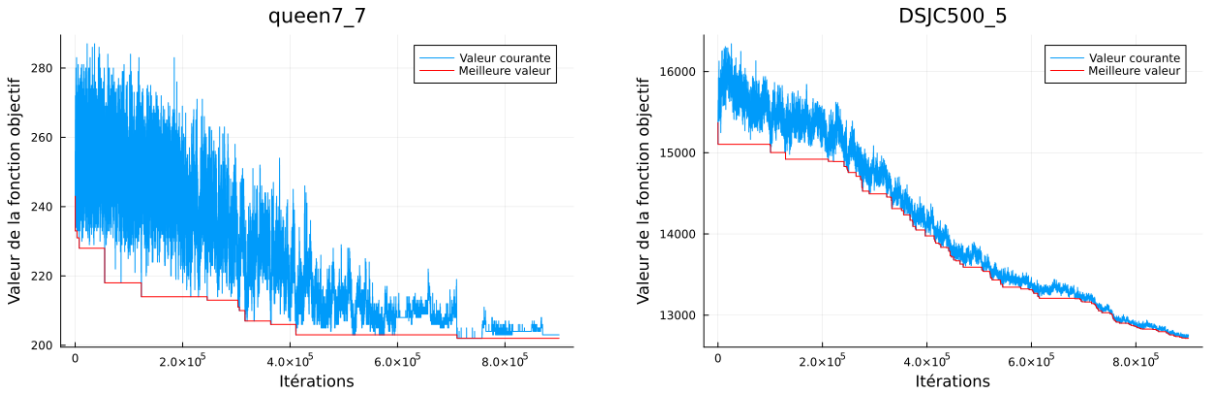
4.4 Visualisation du recuit simulé

Dans les figures 1 et 2 ci-dessous, on observe l'évolution de la recherche au fur et à mesure que la température baisse. En rouge la meilleure solution connue actuelle (la première étant la solution donnée par le glouton DSATUR) et en bleu l'ensemble des solutions parcourues par l'algorithme. Le résultat est très visuel et très cohérent avec le comportement attendu d'un recuit simulé. Lorsque la température est élevée, la qualité des solutions est faible mais il y a une grande diversité de solutions différentes. Au fur et à mesure que la température baisse, on affine la qualité de la solution et on réduit l'exploration pour augmenter l'intensification. L'ensemble des figures pour chaque instance sont disponibles en annexe A.2.



(a) (b)

FIGURE 1 – Exécution du voisinage 'OneMove'

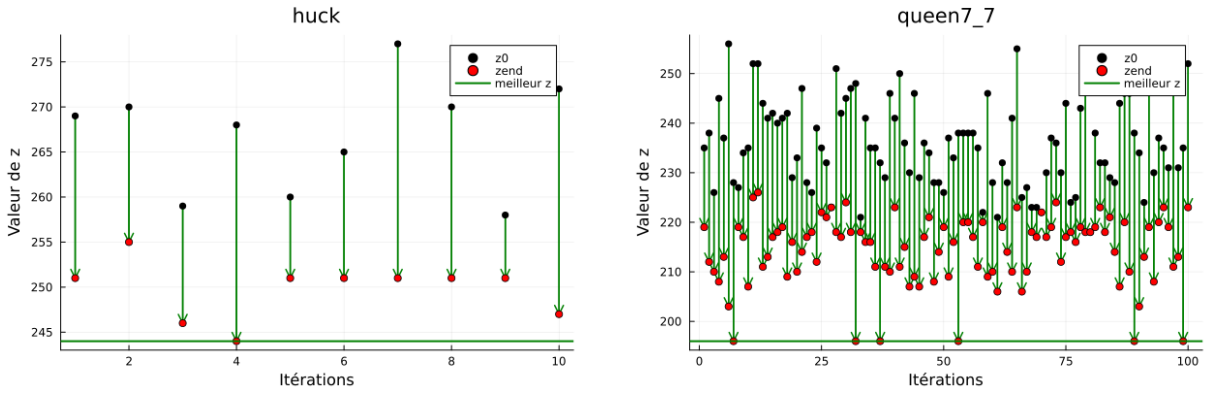


(a) (b)

FIGURE 2 – Exécution du voisinage 'Exchange Swap'

4.5 Visualisation du multi-start

Le multi-start lui peut être visualisé de la manière suivante. Pour chaque instance en abscisse, on représente la solution initiale par un point noir, et l'amélioration de celle-ci par le recuit simulé jusqu'au point rouge.



(a) (b)

FIGURE 3 – Visualisation du multi-start

4.6 Analyse des résultats

Apport du recuit simulé. Le recuit améliore la solution DSATUR sur toutes les instances. Le gain est important sur les petites instances : avec le *one-move*, le meilleur run passe de 304 à 243 sur *huck* (−20 %), de 243 à 196 sur *queen7.7* (−19 %) et de 352 à 290 sur *anna* (−18 %). Il est beaucoup plus faible sur les grandes instances (−5 % sur *le450.25c*, −3,5 % sur *DSJC500.5*) et nul sur *DSJC1000.9* : les 10 exécutions du *one-move* y renvoient exactement la valeur DSATUR (145 020).

Comparaison des deux voisinages. Aucun voisinage ne domine l'autre. En moyenne sur 10 exécutions, le *swap* est meilleur sur 5 instances (*queen7.7*, *queen8.12*, *games120*, *DSJC500.5*, *DSJC1000.9*) et le *one-move* sur 5 autres (*queen5.5*, *huck*, *anna*, *david*, *le450.25c*) ; *myciel4* est résolue (45) par les deux. L'avantage du *swap* est net sur les grands graphes denses (*DSJC500.5* : 13 503 contre 14 980 ; *DSJC1000.9* : 129 058 contre 145 020), celui du *one-move* sur les graphes plus simples (*anna* : 294 contre 349 et *david* : 252 contre 290).

Temps de calcul. Le *one-move* s'exécute en 0,1 à 0,5 s sur toutes les instances sauf *DSJC1000.9* (4,5 s), le *swap* en 1 à 3 s sur les petites instances et en 7 à 12 s sur les trois plus grandes. Cette comparaison est biaisée : le *one-move* a bénéficié de l'évaluation incrémentale de l'annexe A.1, qui a fait passer son temps de 2 min 31 s à 0,5 s sur *DSJC500.5* et de 17 min 46 s à 4,5 s sur *DSJC1000.9*, alors que le *swap* n'a pas reçu d'optimisation équivalente. La comparaison de qualité à nombre d'itérations égal reste valable, mais pas celle des temps de calcul.

Écart aux meilleurs résultats de la littérature. Par rapport aux meilleures bornes supérieures publiées [5], le multi-start *one-move* atteint la borne sur 4 instances (dont l'optimum prouvé sur 3) et s'en écarte de moins de 2 % sur 4 autres. Sur les trois grandes instances, l'écart reste de 19 % (*le450.25c*), 33 % (*DSJC500.5*) et 35 % (*DSJC1000.9*) pour le multi-start *one-move*. Le *swap*, même avec un seul départ, fait mieux sur les deux instances *DSJC* (22 % et 24 % pour le meilleur des 10 runs), mais pas sur *le450.25c* (23 %).

5 Conclusion et perspectives

L'objectif était de proposer une approche fidèle à la formulation théorique du recuit simulé. Le couple DSATUR + recuit simulé y répond. Avec le multi-start one-move (voir 5) on atteint la meilleure borne supérieure connue sur 4 instances, et on est à moins de 2% sur 4 autres instances. Les grosses instances que sont *le450.25c*, *DSJC500.5* et *DSJC1000.9* restent néanmoins plus difficiles.

Les deux voisinages proposent des résultats différents et sont en fait assez complémentaires. Le one-move gagne sur les petites instances et les graphes creux. Tandis que sur les grands graphes, il n'améliore que très peu voire pas du tout la solution initiale de DSATUR. Le swap quant à lui gagne sur ces graphes grands et denses mais peine à explorer. Le fait de rester dans l'espace réalisable le bloque parfois pour obtenir de meilleurs résultats (il reste coincé à 78 sur *queen5.5* par exemple).

Cependant, encore une fois, le *one-move* a fait l'objet d'un réel travail d'optimisation, avant donc de comparer plus en profondeur ces deux voisinages il faut s'assurer que le *swap* fasse l'objet aussi d'une optimisation profonde.

Le recuit "pur" montre donc ses limites sur les grandes instances et plusieurs pistes d'amélioration me viennent en tête :

- combiner les deux voisinages, à la manière d'un VNS ou par simple alternance ;
- affiner la valeur de la pénalité p, notamment sur les grandes instances, où $p = n + 1$ est peut-être trop élevée ;
- calibrer la température séparément pour chaque voisinage ;
- relancer l'exploration par « réchauffage » ;
- hybrider avec une population ou une recherche tabou.

Bibliographie

[1] Una Benlic et Jin-Kao Hao. “A study of breakout local search for the minimum sum coloring problem”. In : *Simulated Evolution and Learning (SEAL 2012)*. T. 7673. Lecture Notes in Computer Science. Springer, 2012, p. 128-137.

[2] Daniel Brélaz. “New methods to color the vertices of a graph”. In : *Communications of the ACM* 22.4 (1979), p. 251-256. doi : 10.1145/359094.359101.

[3] M. Chams, A. Hertz et D. de Werra. “Some experiments with simulated annealing for coloring graphs”. In : *European Journal of Operational Research* 32.2 (1987), p. 260-266. doi : 10.1016/S0377-2217(87)80148-0.

[4] Diego Delle Donne et al. “A branch-and-price algorithm for the minimum sum coloring problem”. In : *Discrete Applied Mathematics* (2021).

[5] Yan Jin, Jean-Philippe Hamiez et Jin-Kao Hao. “Algorithms for the minimum sum coloring problem : a review”. In : *Artificial Intelligence Review* 47.3 (2016). Version en ligne disponible depuis 2016, p. 367-394. doi : 10.1007/s10462-016-9485-7.

[6] Yan Jin et Jin-Kao Hao. “Hybrid evolutionary search for the minimum sum coloring problem of graphs”. In : *Information Sciences* 352-353 (2016). Algorithme HESA, référence pour les meilleures bornes supérieures connues sur de nombreuses instances DIMACS, p. 15-34. doi : 10.1016/j.ins.2016.03.008.

[7] Yan Jin, Jin-Kao Hao et Jean-Philippe Hamiez. “A memetic algorithm for the minimum sum coloring problem”. In : *Computers & Operations Research* 43 (2014), p. 318-327.

[8] S. Kirkpatrick, C. D. Gelatt et M. P. Vecchi. “Optimization by Simulated Annealing”. In : *Science* 220.4598 (1983), p. 671-680. doi : 10.1126/science.220.4598.671.

[9] Ewa Kubicka et Allen J. Schwenk. “An introduction to chromatic sums”. In : *Proceedings of the 17th Conference on ACM Annual Computer Science Conference (CSC ’89)*. Article fondateur qui introduit la somme chromatique, ancêtre du MSCP. New York, NY, USA : ACM, 1989, p. 39-45.

[10] Clément Lecat et al. “Exact methods for the minimum sum coloring problem”. In : août 2015.

[11] Maël Minot, Samba Ndojh Ndiaye et Christine Solnon. “Combining CP and ILP in a tree decomposition of bounded height for the sum colouring problem”. In : 2016.

[12] Qinghua Wu et Jin-Kao Hao. “An effective heuristic algorithm for sum coloring of graphs”. In : *Computers & Operations Research* 39.7 (2012). Algorithme EXSCOL, une des meilleures métaheuristiques de l’état de l’art pour le MSCP, p. 1593-1600. doi : 10.1016/j.cor.2011.09.010.

A Annexes

A.1 Recuit simulé one-move à évaluation incrémentale

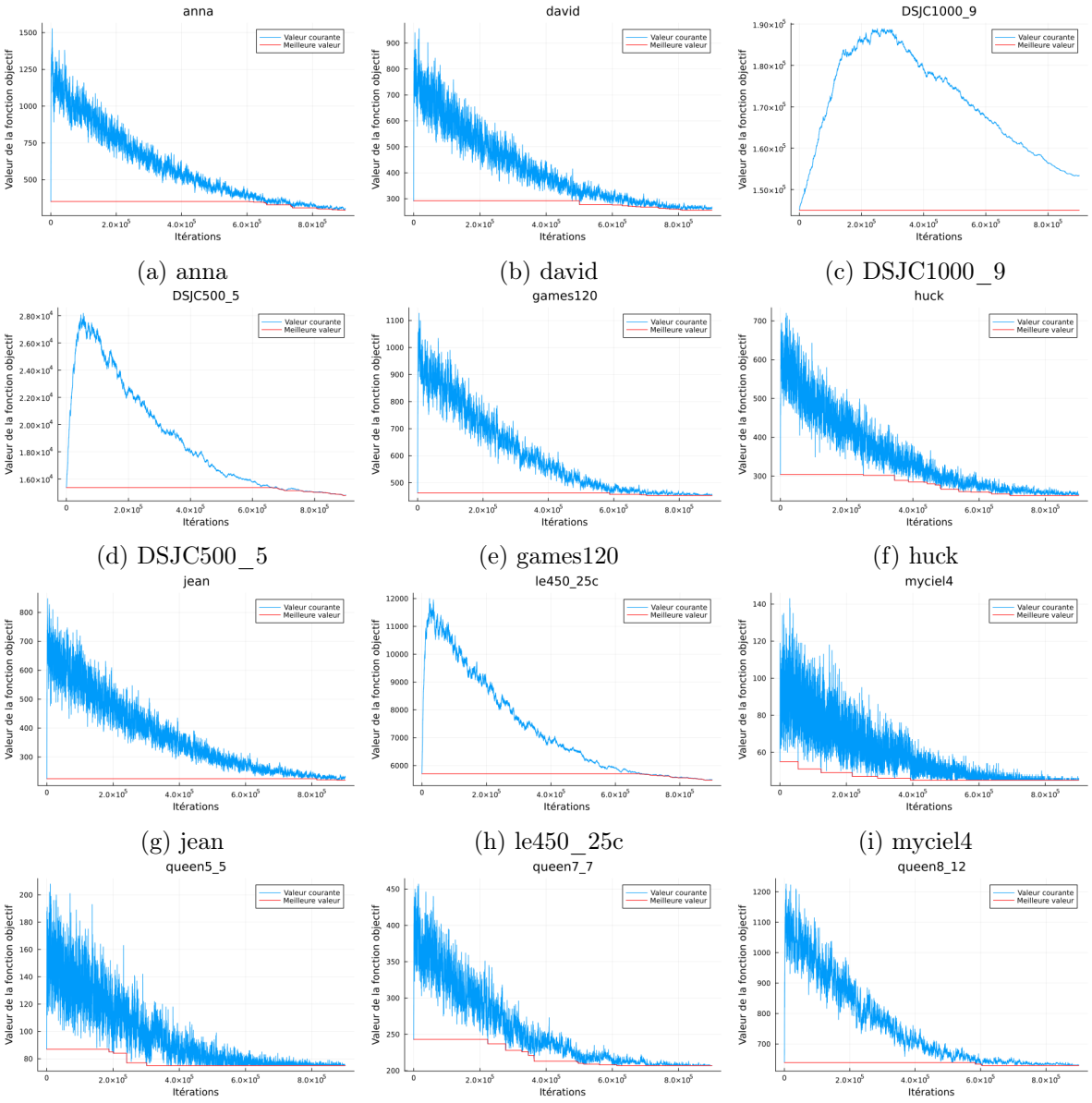
Algorithm 6 Recuit simulé one-move avec évaluation incrémentale (`recuit_rapide.jl`)

Require: x_0, $adjOf$, T_0, $T_{\min}$, p, α, $nbIter$
Ensure: meilleure coloration trouvée x_{best} et sa valeur f_{best}

$x \leftarrow x_0$
$conf[v] \leftarrow |\{u \in adjOf[v] : x[u] = x[v]\}|$ pour tout $v \in V$ ▷ voisins de même couleur
$C \leftarrow \{v \in V : conf[v] > 0\}$ ▷ sommets en conflit
$c_{\max} \leftarrow \max_v x[v]$
$f \leftarrow \sum_v x[v] + p \cdot \frac{1}{2} \sum_v conf[v]$ ▷ valeur pénalisée courante
$x_{\text{best}} \leftarrow x, \quad f_{\text{best}} \leftarrow f, \quad T \leftarrow T_0$
while $T > T_{\min}$ **do**
  for $i = 1$ **à** $nbIter$ **do**
    $v \leftarrow$ sommet tiré uniformément dans C (dans V si $C = \emptyset$)
    $c_{old} \leftarrow x[v]$
    $c_{new} \leftarrow$ couleur tirée uniformément dans $\{1, \ldots, c_{\max} + 1\} \setminus \{c_{old}\}$
    $a_{old} \leftarrow conf[v]$
    $a_{new} \leftarrow |\{u \in adjOf[v] : x[u] = c_{new}\}|$
    $\Delta \leftarrow (c_{new} - c_{old}) + p\,(a_{new} - a_{old})$
    if $\Delta \leq 0$ **ou** rand() $< e^{-\Delta/T}$ **then**
      for each $u \in adjOf[v]$ **do** ▷ mise à jour des voisins de v
        if $x[u] = c_{old}$ **then**
          $conf[u] \leftarrow conf[u] - 1$
          retirer u de C si $conf[u] = 0$
        else if $x[u] = c_{new}$ **then**
          $conf[u] \leftarrow conf[u] + 1$
          ajouter u à C si $conf[u] = 1$
        end if
      end for
      $conf[v] \leftarrow a_{new}$; mettre à jour l'appartenance de v à C
      $x[v] \leftarrow c_{new}$; mettre à jour $c_{\max}$
      $f \leftarrow f + \Delta$
      if $f < f_{\text{best}}$ **then**
        $x_{\text{best}} \leftarrow x, \quad f_{\text{best}} \leftarrow f$
      end if
    end if
  end for
  $T \leftarrow \alpha \cdot T$
end while
return $x_{\text{best}}, f_{\text{best}}$

A.2 Évolution de la fonction objectif par instance

Voisinage « onemove »



(a) anna (b) david (c) DSJC1000_9

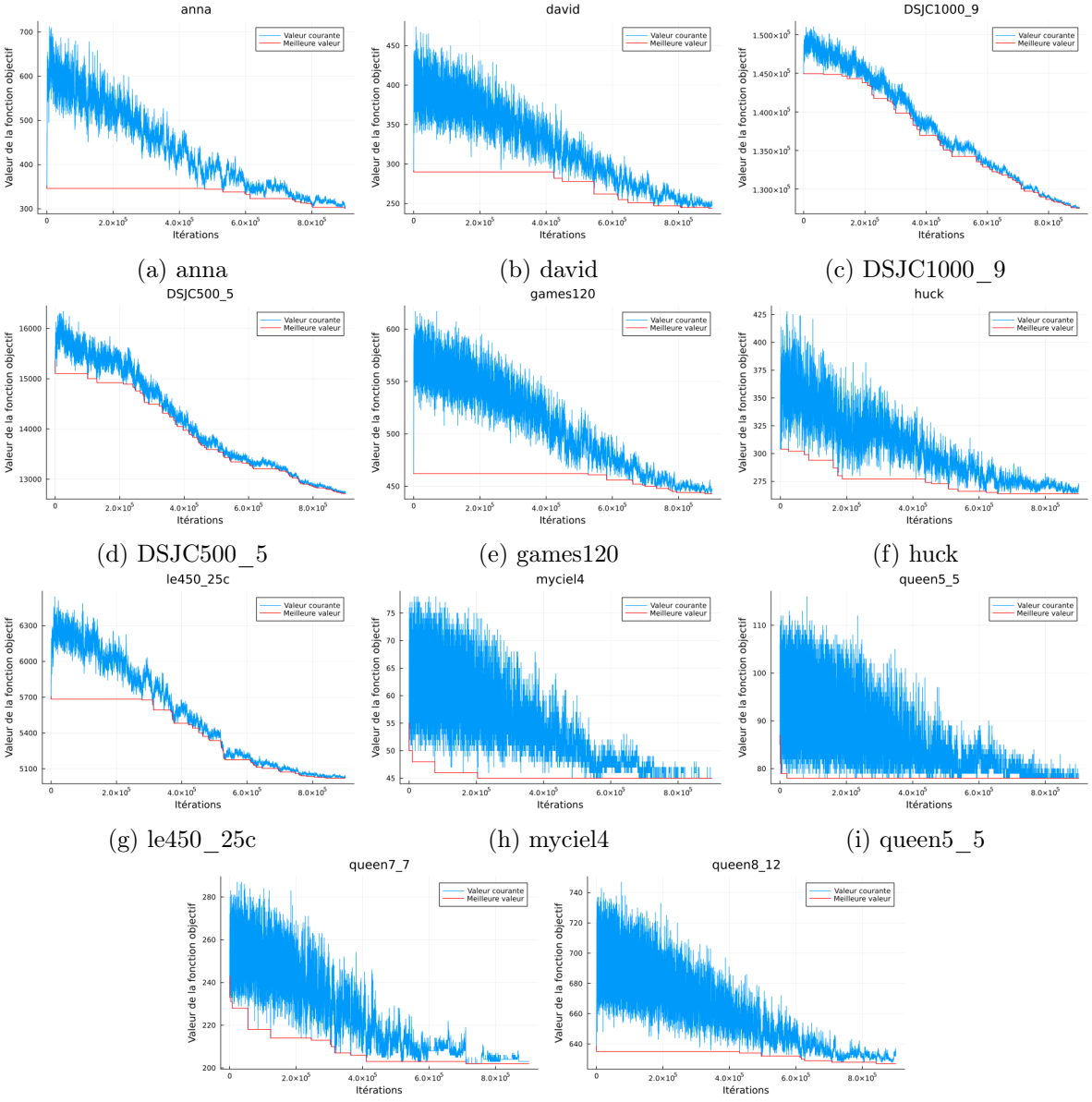
(d) DSJC500_5 (e) games120 (f) huck

(g) jean (h) le450_25c (i) myciel4

(j) queen5_5 (k) queen7_7 (l) queen8_12

FIGURE 4 – Évolution de la fonction objectif – voisinage onemove

Voisinage « swap »



(a) anna (b) david (c) DSJC1000_9

(d) DSJC500_5 (e) games120 (f) huck

(g) le450_25c (h) myciel4 (i) queen5_5

(j) queen7_7 (k) queen8_12

FIGURE 5 – Évolution de la fonction objectif – voisinage swap